МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 10

Тема: Поведенческие паттерны: Наблюдатель, состояние

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы поведенческих паттернов проектирования «Наблюдатель» (Observer) и «Состояние» (State) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два поведенческих паттерна из списка: «Наблюдатель», «Состояние».

В соответствии с заданием были выбраны паттерны «Наблюдатель» (Observer) и «Состояние» (State).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу поведенческих паттернов: наблюдатель позволяет UI автоматически реагировать на изменения в PasswordManager, а состояние - менять поведение кнопок и сетки в зависимости от жизненного цикла пароля.

1. Описание паттерна «Наблюдатель»

Паттерн «Наблюдатель» (Observer) - это поведенческий паттерн проектирования, который создаёт механизм подписки, позволяющий одним объектам следить и реагировать на события, происходящие в других объектах.

Данный паттерн применяется, когда изменение состояния одного объекта требует изменения других объектов, и когда нужно ослабить связанность между субъектом и наблюдателями.

В разработанном приложении паттерн «Наблюдатель» используется для автоматического обновления интерфейса при изменении состояния пароля. В программе определены роли:

- PasswordManager - субъект (Subject). При создании пароля вызывает _notify("password_created", sequence), при сбросе - _notify("password_reset");

- StateObserver - интерфейс наблюдателя с методом on_state_changed(event, info);

- AppWindow - конкретный наблюдатель, который подписывается на события через password_manager.subscribe(self).

При каждом событии AppWindow обновляет своё состояние (NoPasswordState / PasswordSetState) и перерисовывает интерфейс. Клиентский код (AppWindow) не опрашивает PasswordManager, а получает уведомления автоматически.

Таким образом, PasswordManager не знает, кто на него подписан, и может иметь любое количество наблюдателей.

2. Описание паттерна «Состояние»

Паттерн «Состояние» (State) - это поведенческий паттерн проектирования, который позволяет объекту менять своё поведение в зависимости от внутреннего состояния. Внешне это выглядит так, как будто объект меняет свой класс.

Данный паттерн применяется, когда поведение объекта зависит от его состояния и должно изменяться во время выполнения.

В разработанном приложении паттерн «Состояние» используется для управления жизненным циклом пароля. В программе определены роли:

- AppState - абстрактное состояние с методами can_login(), can_create(), get_primary_text(mode), get_status_text(), get_name(), get_color();

- NoPasswordState - пароль не установлен: вход запрещён, можно только создать;

- PasswordSetState - пароль установлен: доступны и вход, и перезапись;

- BlockedState - блокировка после 5 неверных попыток на 30 секунд: все действия запрещены.

Переходы между состояниями:

- NoPasswordState → PasswordSetState при создании пароля (по событию от наблюдателя);

- PasswordSetState → NoPasswordState при сбросе пароля;

- PasswordSetState → BlockedState после 5 неверных попыток входа;

- BlockedState → PasswordSetState после истечения таймера.

UI автоматически подстраивается: бейдж состояния над сеткой показывает название и цвет, основная кнопка меняет текст, кнопки режимов блокируются.

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит:

- бейдж текущего состояния (зелёный - «Пароль установлен», жёлтый - «Нет пароля», красный - «Заблокировано»);
- сетку из девяти скруглённых ячеек;
- текущую последовательность и счётчик неудачных попыток;
- основную кнопку («Создать пароль» / «Войти» / «Сохранить пароль» / «Заблокировано»);
- кнопки «Очистить» и «Сбросить сохранённый пароль»;
- переключатель режимов «Вход» / «Создать пароль».

К паролю предъявляются требования:

- длина от 4 до 9 ячеек;
- отсутствие повторов;
- ячейки в диапазоне 1–9.

Дополнительно реализована защита от подбора: после 5 неверных попыток приложение переходит в BlockedState на 30 секунд. При этом все действия блокируются, а основная кнопка отображает оставшееся время.

4. Диаграмма классов

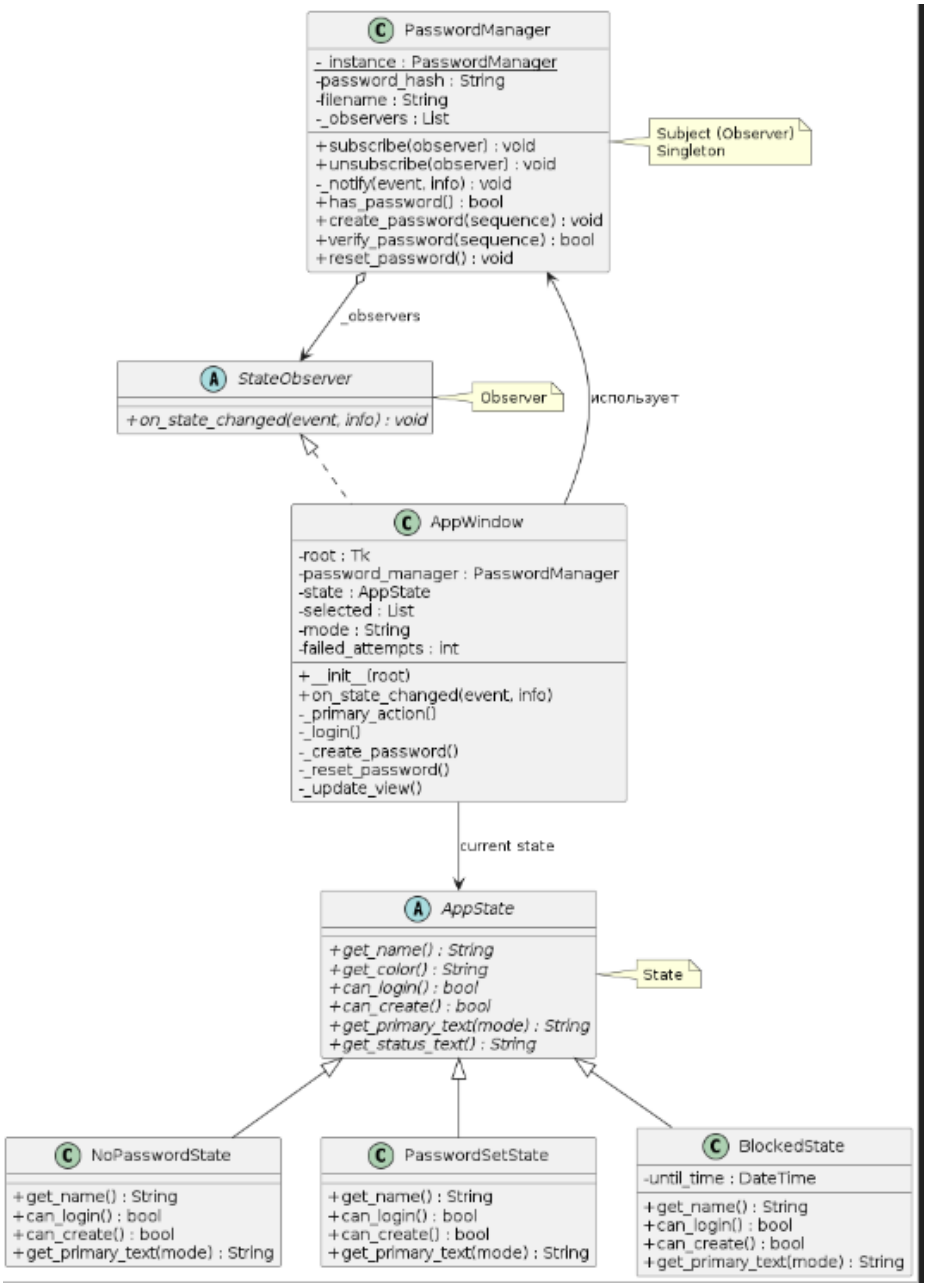


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит:

- PasswordManager и StateObserver - паттерн «Наблюдатель»;

- AppState, NoPasswordState, PasswordSetState, BlockedState - паттерн «Состояние»;

- AppWindow - основной класс интерфейса (наблюдатель + клиент состояний).

5. Реализация паттерна «Наблюдатель»

Интерфейс наблюдателя:

```
class StateObserver(ABC):
    @abstractmethod
    def on_state_changed(self, event, info=None):
        pass
```

Субъект - PasswordManager:

```
def subscribe(self, observer):
    if observer not in self._observers:
        self._observers.append(observer)

def _notify(self, event, info=None):
    for observer in list(self._observers):
        observer.on_state_changed(event, info)

def create_password(self, sequence):
    self.password_hash = hash_sequence(sequence)
    self._save()
    self._notify("password_created", sequence)

def reset_password(self):
    self.password_hash = None
    if os.path.exists(self.filename):
        os.remove(self.filename)
    self._notify("password_reset")
```

Конкретный наблюдатель - AppWindow:

```
def on_state_changed(self, event, info=None):
    if event == "password_created":
        self.state = PasswordSetState()
        self.failed_attempts = 0
    elif event == "password_reset":
        self.state = NoPasswordState()
```

```
            self.failed_attempts = 0
        self._update_view()
```

UI автоматически обновляется без прямого вызова из PasswordManager.

6. Реализация паттерна «Состояние»

Абстрактное состояние:

```
class AppState(ABC):
    @abstractmethod
    def get_name(self): pass

    @abstractmethod
    def get_color(self): pass

    @abstractmethod
    def can_login(self): pass

    @abstractmethod
    def can_create(self): pass

    @abstractmethod
    def get_primary_text(self, mode): pass

    @abstractmethod
    def get_status_text(self): pass
```

Конкретные состояния:

```
class NoPasswordState(AppState):
    def get_name(self): return "Нет пароля"
    def get_color(self): return "#ffaa00"
    def can_login(self): return False
    def can_create(self): return True
    def get_primary_text(self, mode): return "Создать пароль"
```

```
    def get_status_text(self):
        return "Пароль не установлен. Сначала создайте пароль."

class PasswordSetState(AppState):
    def get_name(self): return "Пароль установлен"
    def get_color(self): return "#4caf50"
    def can_login(self): return True
    def can_create(self): return True
    def get_primary_text(self, mode):
        return "Войти" if mode == "login" else "Сохранить пароль"
    def get_status_text(self):
        return "Пароль установлен. Можно войти или перезаписать."

class BlockedState(AppState):
    def __init__(self, until_time):
        self.until_time = until_time
    def get_name(self): return "Заблокировано"
    def get_color(self): return "#ff4444"
    def can_login(self): return False
    def can_create(self): return False
    def get_primary_text(self, mode):
        remaining = int((self.until_time - datetime.now()).total_seconds())
        return f"Заблокировано ({remaining} с)"
    def get_status_text(self):
        remaining = int((self.until_time - datetime.now()).total_seconds())
        return f"Слишком много неверных попыток. Подождите
{remaining} с."
```

В UI действия проверяют состояние:

```
def _login(self):
    if not self.state.can_login():
```

```
            self.status_label.config(text="Вход недоступен.", fg="#ff4444")
            return
        ...
    def _select_cell(self, num):
        if not self.state.can_login() and not self.state.can_create():
            self.status_label.config(text="Доступ заблокирован.", fg="#ff4444")
            return
        ...
```

При неудачных попытках:

```
self.failed_attempts += 1
if self.failed_attempts >= self.MAX_ATTEMPTS:
    until = datetime.now() + timedelta(seconds=self.BLOCK_SECONDS)
    self.state = BlockedState(until)
```

7. Результаты работы программы

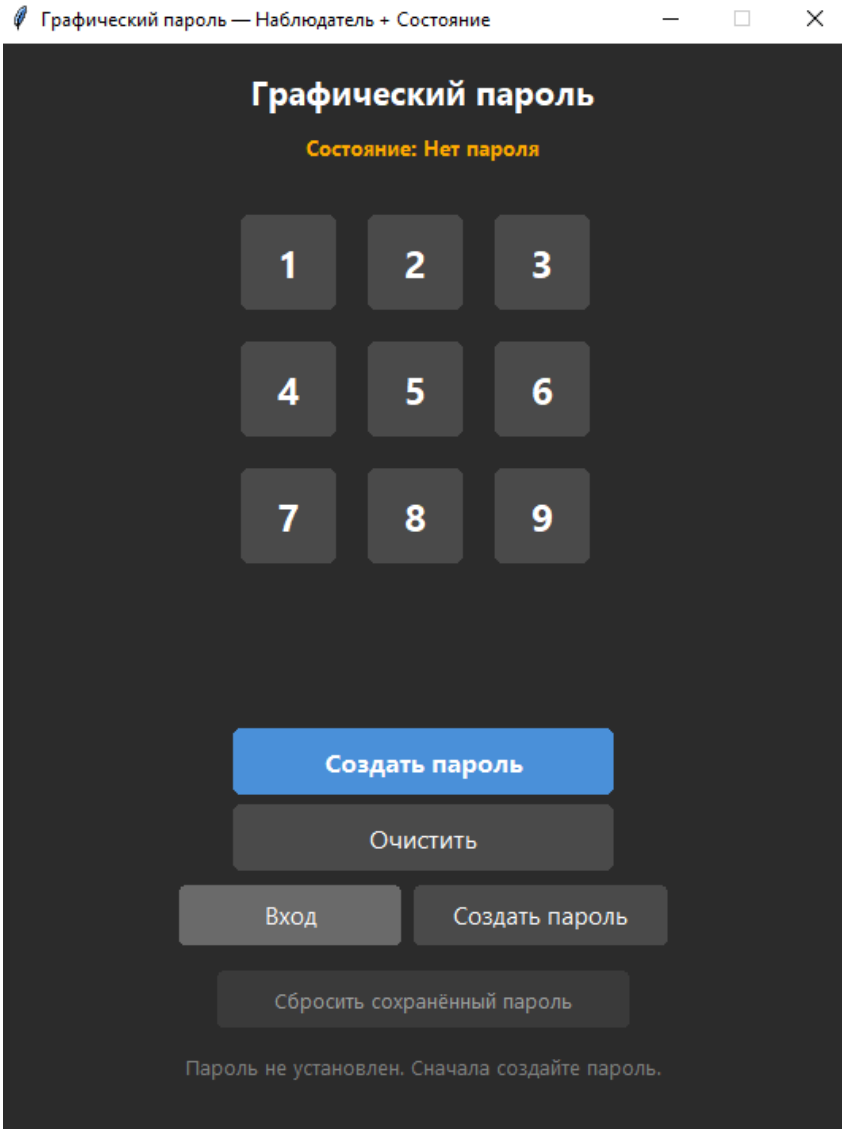


Рисунок 2 - Главное окно приложения (состояние «Нет пароля»)

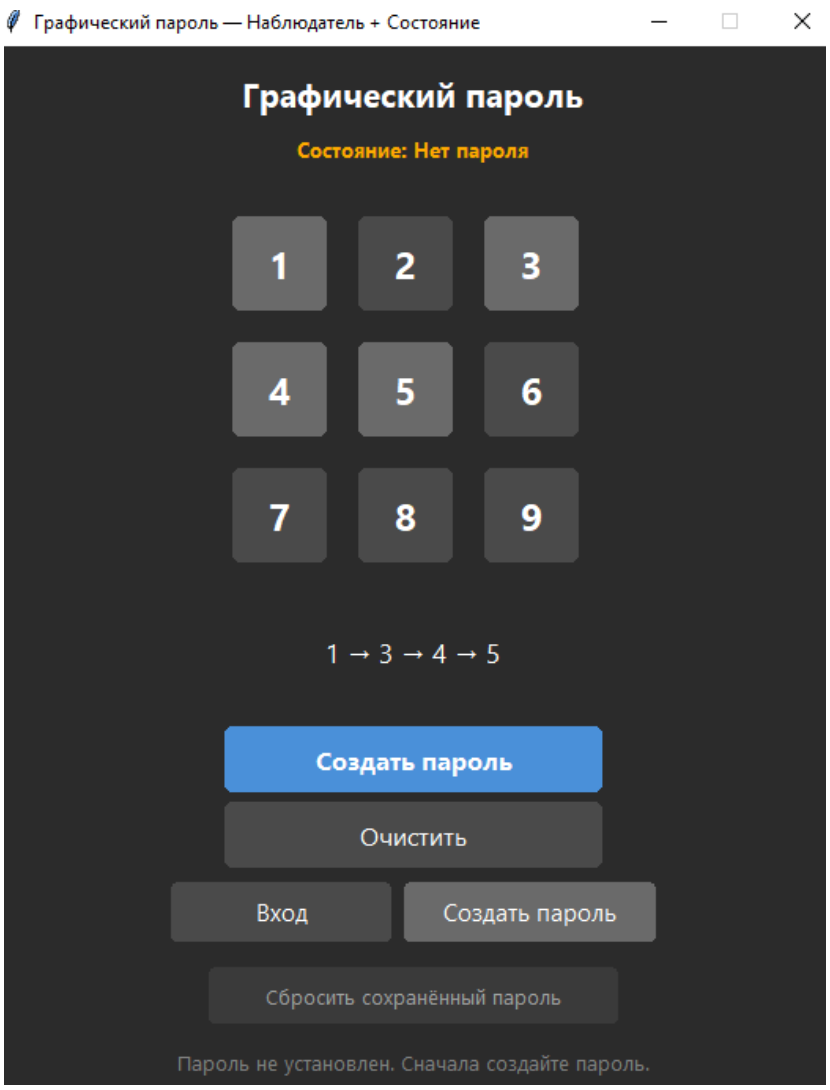


Рисунок 3 - Создание пароля (переход в PasswordSetState)

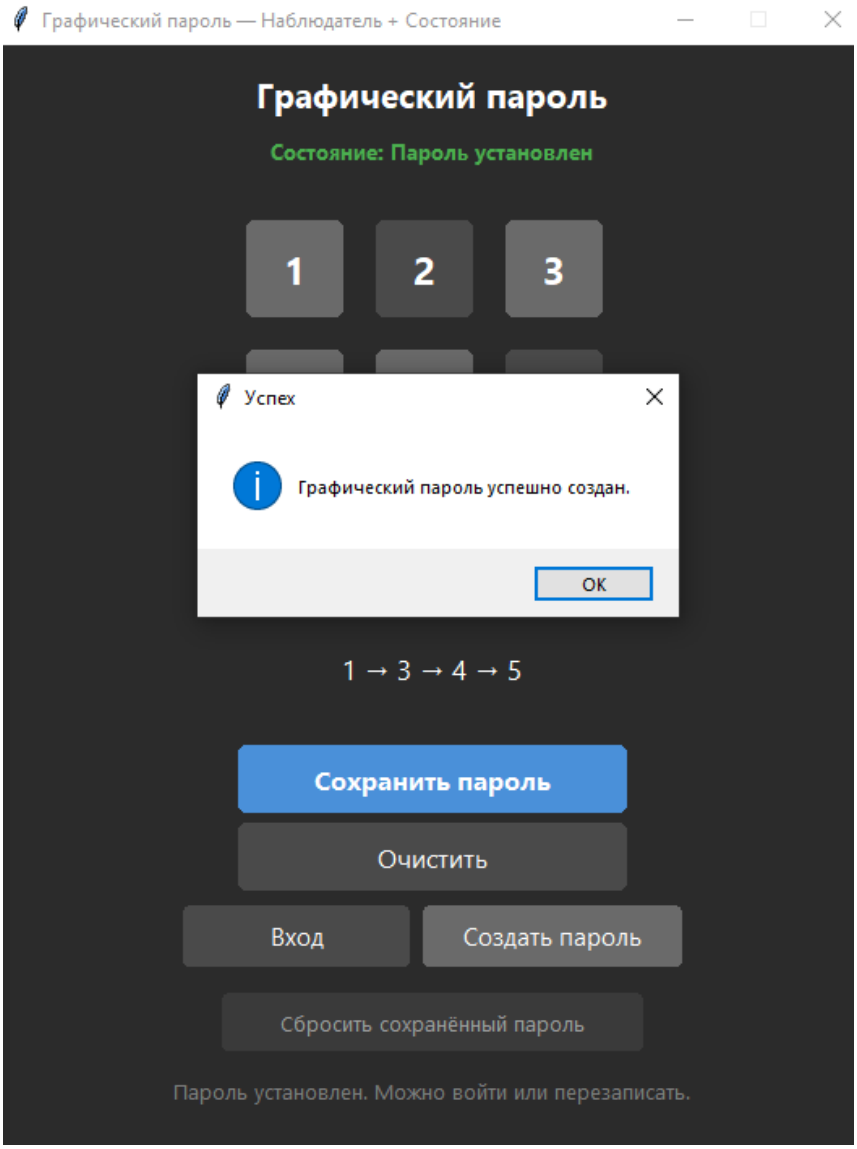


Рисунок 4 - Успешный вход (бейдж «Пароль установлен»)

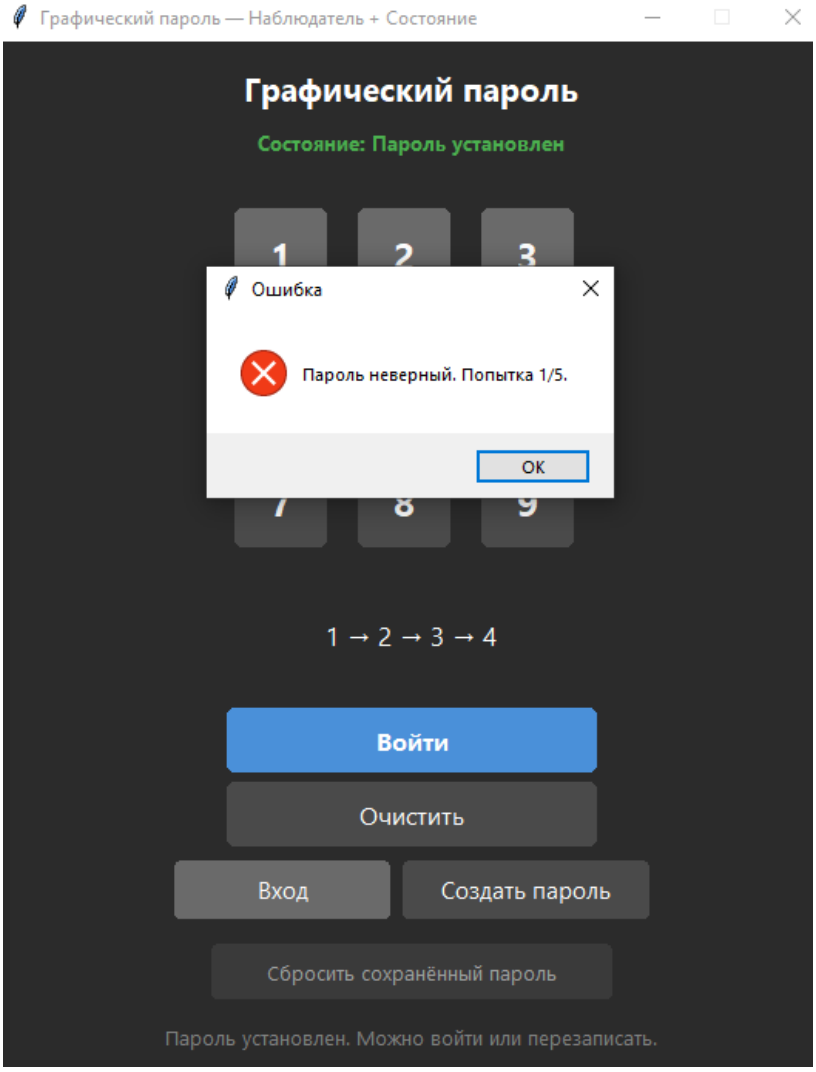


Рисунок 5 - Неверный пароль (счётчик попыток 1/5)

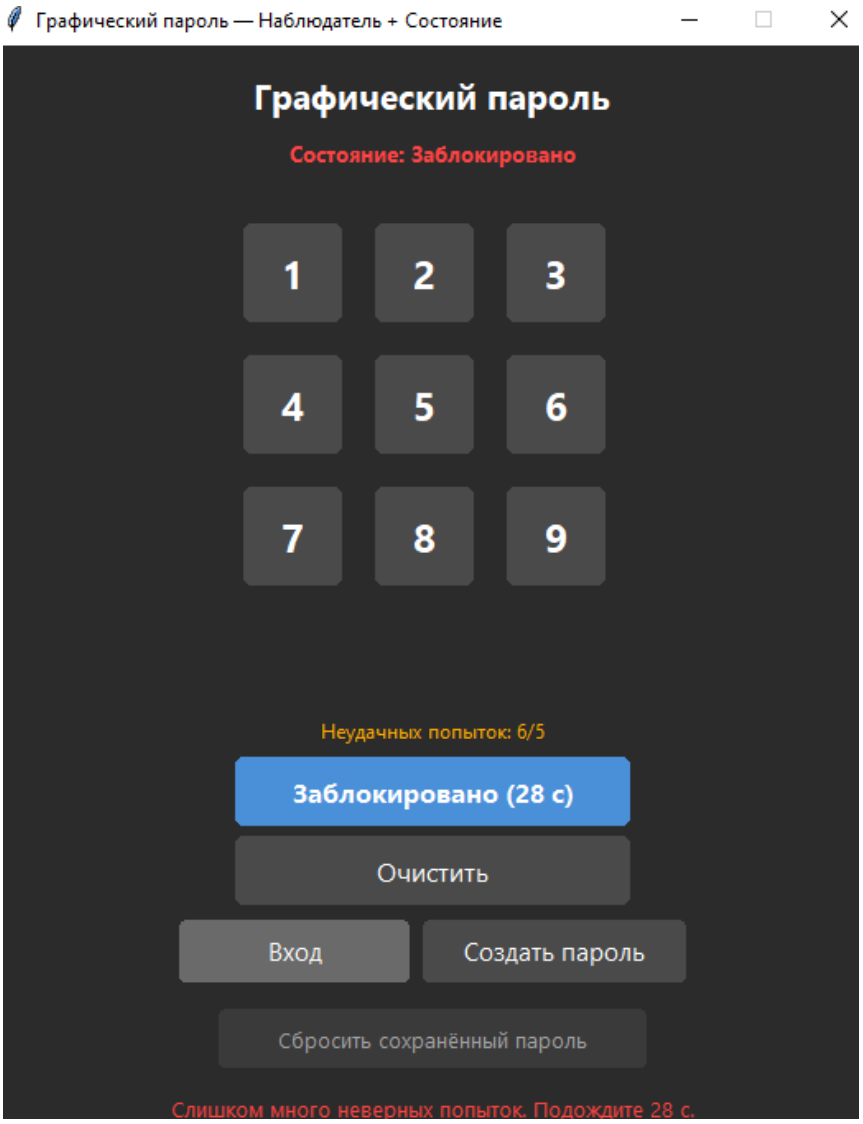


Рисунок 6 - Блокировка после 5 неверных попыток (BlockedState, красный бейдж)

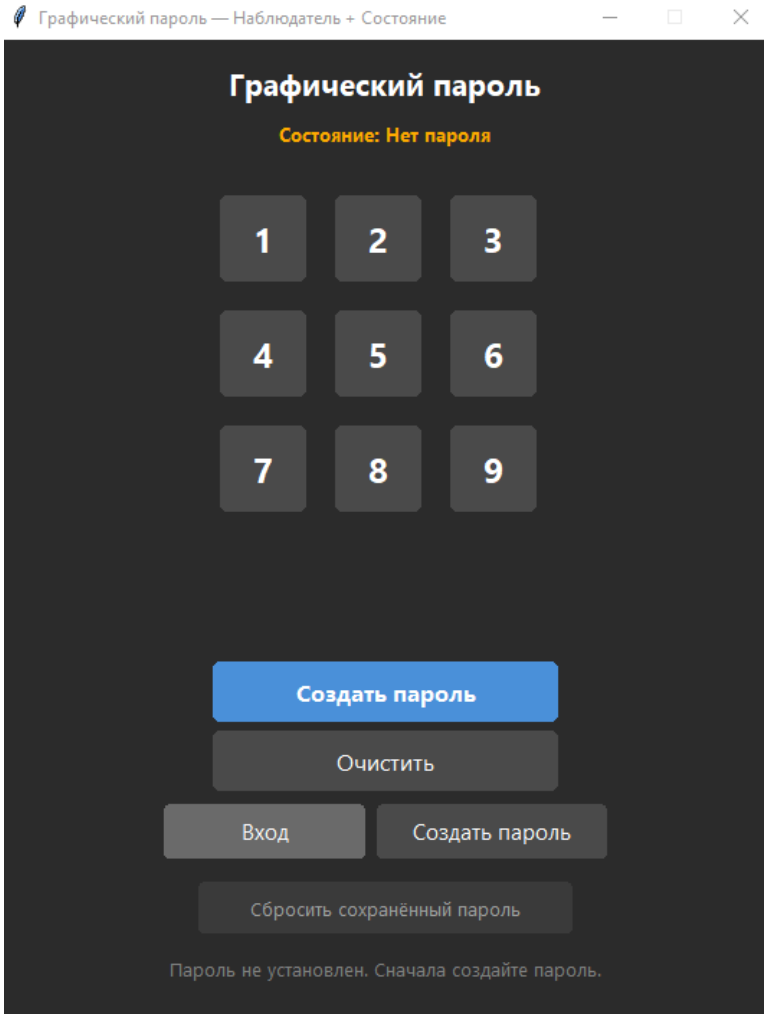


Рисунок 7 - Обновление UI через наблюдателя (после создания/сброса пароля бейдж меняется автоматически)

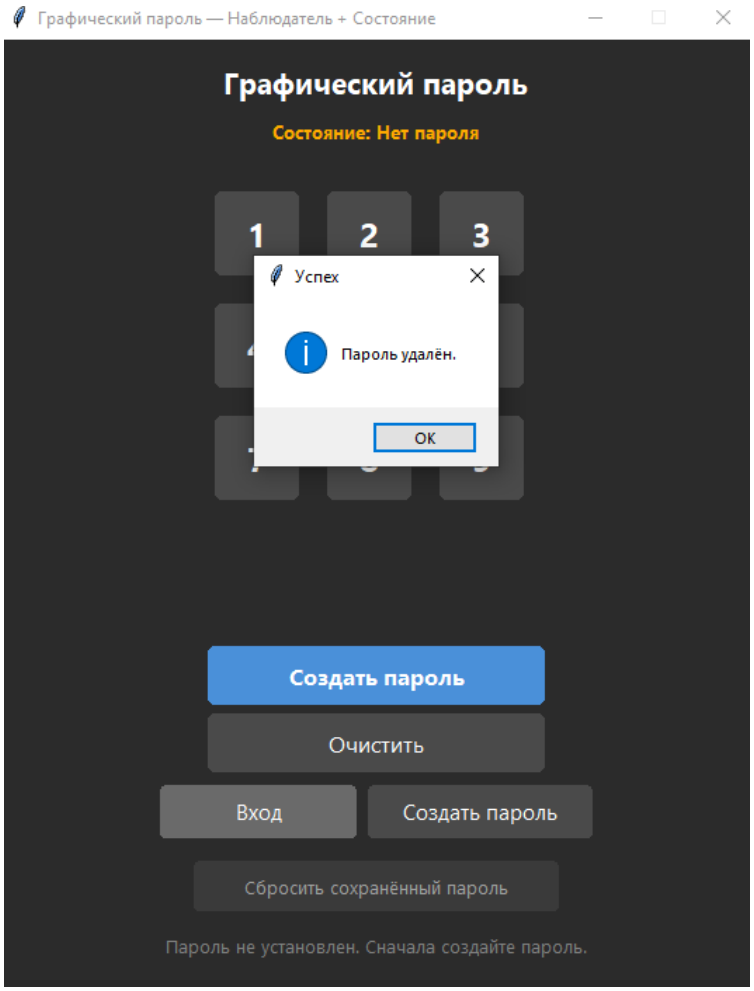


Рисунок 8 - Сброс пароля (переход в NoPasswordState)

8. Исходный код

Основные элементы программы:

- StateObserver - интерфейс наблюдателя;
- PasswordManager - субъект (Singleton), уведомляющий наблюдателей;
- AppState, NoPasswordState, PasswordSetState, BlockedState - состояния;
- AppWindow - основной класс интерфейса (наблюдатель + клиент состояний);
- RoundedButton, rounded_rect() - скруглённые кнопки и ячейки;
- hash_sequence(), validate_sequence() - вспомогательные функции.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Были изучены и реализованы два поведенческих паттерна проектирования: «Наблюдатель» (Observer) и «Состояние» (State). Паттерн «Наблюдатель» применён через подписку AppWindow на события PasswordManager, что позволило автоматически обновлять интерфейс при создании и сбросе пароля. Паттерн «Состояние» применён для управления жизненным циклом пароля: приложение переключается между NoPasswordState, PasswordSetState и BlockedState, и поведение кнопок и сетки автоматически подстраивается под текущее состояние. В результате получена работоспособная программа, демонстрирующая практическое применение поведенческих паттернов при разработке прикладного программного обеспечения.